

Contenus pédagogiques · progression Python

Quatre fiches de cours

Quatre fiches conçues comme une progression et non comme des notions isolées. Chacune prépare la suivante, et la dernière réunit les trois autres sur une donnée réelle : un fichier de journal d'authentification.

La progression

- **Fiche 01 — Les boucles.** Répéter une action sur une séquence. Sans elle, rien de ce qui suit n'est possible.
- **Fiche 02 — Les listes.** Ranger les données que la boucle va parcourir, et apprendre à filtrer par construction.
- **Fiche 03 — Les dictionnaires.** Passer de « quels éléments » à « combien de fois ». Le motif de comptage est le cœur du cours.
- **Fiche 04 — Lire un fichier de journal.** Les trois notions réunies sur un fichier réel, avec un programme d'analyse complet en vingt lignes.

Parti pris de rédaction

- Chaque notion s'ouvre sur le problème qu'elle résout, jamais sur sa définition.
- Les exemples de code sont tirés d'un contexte cybersécurité, pour que la notion arrive avec une raison d'exister.
- Une rubrique « erreurs fréquentes » par fiche : ce sont elles qui bloquent les débutants, plus que la notion elle-même.
- Les simplifications pédagogiques sont signalées comme telles, pour ne pas laisser croire qu'un exercice est un modèle de production.

Fiche 01 • Programmation • Python

Les boucles : répéter sans se répéter

Trois mots de passe se vérifient à la main. Quarante mille, non. La boucle est ce qui sépare un exercice d'école d'un outil qui sert à quelque chose.

Niveau	Débutant — NSI première, BTS SIO première année
Prérequis	Variables, types, conditions (if / else)
Durée	1 h 30 avec exercices

Objectifs d'apprentissage

- Répéter une instruction sans la recopier
- Parcourir une séquence élément par élément avec **for**
- Utiliser **while** quand le nombre de tours n'est pas connu à l'avance
- Interrompre ou passer un tour avec **break** et **continue**
- Reconnaître et éviter une boucle infinie

Le problème que résout une boucle

Un administrateur veut vérifier que les mots de passe de ses utilisateurs font au moins huit caractères. Avec trois comptes, la version manuelle passe encore :

```
mdp1 = "azerty"
mdp2 = "Tr0ub4dor&3"
mdp3 = "1234"

if len(mdp1) < 8:
    print("Trop court :", mdp1)
if len(mdp2) < 8:
    print("Trop court :", mdp2)
if len(mdp3) < 8:
    print("Trop court :", mdp3)
```

Avec quarante mille comptes, cette approche n'existe plus. Et surtout, le code ne dit pas ce qu'on veut faire — il dit trois fois la même chose. Une boucle exprime l'intention réelle : *pour chaque mot de passe, vérifie sa longueur.*

La boucle for : parcourir une séquence

for s'utilise quand on sait sur quoi on itère. La variable de boucle prend successivement la valeur de chaque élément.

```
mots_de_passe = ["azerty", "Tr0ub4dor&3", "1234", "correcthorse"]

for mdp in mots_de_passe:
    if len(mdp) < 8:
        print("Trop court :", mdp)
```

Le code est plus court, mais ce n'est pas l'essentiel : il fonctionne à l'identique que la liste contienne quatre entrées ou quarante mille.

Itérer sur des nombres avec range

Quand on veut répéter un nombre de fois donné plutôt que parcourir une collection existante, **range** produit une suite d'entiers. Attention à la borne : elle est exclue.

```
for i in range(5):
    print(i)
# affiche 0, 1, 2, 3, 4 — jamais 5

for tentative in range(1, 4):
    print("Tentative", tentative)
# affiche 1, 2, 3
```

La boucle while : répéter tant qu'une condition tient

while sert quand le nombre de tours n'est pas connu à l'écriture du programme. Le cas typique : on continue tant que quelque chose n'est pas arrivé.

```
essais = 0
connecte = False

while essais < 3 and not connecte:
    mot = demander_mot_de_passe()
    connecte = verifier(mot)
    essais = essais + 1

if not connecte:
    print("Compte verrouillé après 3 échecs")
```

Toute boucle **while** a besoin de trois choses : une variable initialisée avant, une condition qui la teste, et une modification de cette variable à l'intérieur. Si la troisième manque, la boucle ne s'arrête jamais.

Sortir d'une boucle : break et continue

- **break** quitte la boucle immédiatement, sans terminer le tour en cours.
- **continue** abandonne le tour en cours et passe au suivant.

```
journal = ["ok", "ok", "ERREUR", "ok"]

for ligne in journal:
    if ligne == "ERREUR":
        print("Anomalie détectée, arrêt de l'analyse")
        break
    print("Ligne saine :", ligne)
```

Application : détecter les comptes vulnérables

Mise en situation — vous êtes administrateur système. On vous transmet la liste des mots de passe d'un ancien serveur, et vous devez identifier les comptes à faire changer en priorité.

```
base = ["azerty", "Tr0ub4dor&3", "1234", "correcthorsebattery", "admin"]
faibles = []

for mdp in base:
    if len(mdp) < 8:
        faibles.append(mdp)

print("Comptes à traiter :", len(faibles))
for mdp in faibles:
    print(" -", mdp)
```

Remarque de sécurité à faire passer aux élèves : dans un vrai système, on ne manipule jamais des mots de passe en clair. Cet exercice les représente ainsi pour rendre la boucle lisible — c'est une simplification pédagogique, pas un modèle.

Erreurs fréquentes

- Oublier de faire varier la condition dans un **while** : la boucle tourne indéfiniment.
- Croire que **range(5)** va jusqu'à 5. La borne haute est exclue.
- Confondre l'élément et son indice : dans **for x in liste**, x est la valeur, pas la position.
- Modifier une liste pendant qu'on la parcourt — le parcours saute des éléments. Construire une nouvelle liste à la place.
- Oublier l'indentation du bloc : en Python, elle définit ce qui est dans la boucle.

Exercices

- Compter le nombre de lignes contenant le mot « ECHEC » dans une liste de journal.
- Afficher les nombres de 1 à 20 en signalant lesquels sont pairs.
- Simuler trois tentatives de connexion et verrouiller le compte au troisième échec.
- À partir d'une liste d'adresses IP, afficher uniquement celles qui commencent par « 10. ».

Fiche 02 • Programmation • Python

Les listes : ranger, filtrer, nettoyer

Une liste d'utilisateurs autorisés, un relevé d'adresses IP, une suite de tentatives de connexion : en cybersécurité, presque toute donnée arrive sous forme de séquence.

Niveau	Débutant — NSI première, BTS SIO première année
Prérequis	Variables, types, conditions, boucles
Durée	1 h 30 avec exercices

Objectifs d'apprentissage

- Créer une liste et accéder à ses éléments par leur indice
- Ajouter, retirer et remplacer des éléments
- Extraire une portion avec le découpage (slicing)
- Filtrer une liste pour n'en garder qu'une partie
- Comprendre qu'une liste est modifiable, et ce que cela implique

Créer et accéder

Une liste se note entre crochets, éléments séparés par des virgules. Les indices commencent à zéro, et les indices négatifs comptent depuis la fin.

```
utilisateurs = ["alice", "bob", "admin", "invite"]

print(utilisateurs[0])    # alice
print(utilisateurs[2])    # admin
print(utilisateurs[-1])   # invite (le dernier)
print(len(utilisateurs))  # 4
```

Le découpage

Le découpage extrait une portion. Comme pour **range**, la borne de fin est exclue.

```
ips = ["10.0.0.1", "10.0.0.2", "10.0.0.3", "192.168.1.5"]

print(ips[0:2])    # les deux premières
print(ips[2:])     # de la troisième à la fin
print(ips[:-1])    # tout sauf la dernière
```

Modifier une liste

- **append(x)** ajoute un élément à la fin.
- **remove(x)** retire la première occurrence de la valeur x.
- **pop(i)** retire l'élément à la position i et le renvoie.
- **liste[i] = x** remplace l'élément à la position i.

```

autorises = ["alice", "bob", "invite"]

autorises.append("claire")    # ajout
autorises.remove("invite")    # retrait par valeur
autorises[0] = "alice.durand" # remplacement

print(autorises)
# ["alice.durand", "bob", "claire"]

```

Attention : une liste est modifiable

Contrairement à une chaîne de caractères, une liste peut changer après sa création. Deux variables peuvent désigner la *même* liste — modifier l'une modifie l'autre.

```

a = ["alice", "bob"]
b = a          # b désigne la même liste que a
b.append("eve")
print(a)       # ["alice", "bob", "eve"] - a a changé aussi

c = a.copy()   # c est une copie indépendante
c.append("mallory")
print(a)       # inchangée

```

Filtrer : construire plutôt que supprimer

La bonne façon de nettoyer une liste n'est pas d'en retirer les éléments pendant qu'on la parcourt — c'est d'en construire une nouvelle avec ce que l'on veut garder.

```

comptes = ["alice", "test", "bob", "guest", "hacker", "claire"]
interdits = ["test", "guest", "hacker"]

legitimes = []
for nom in comptes:
    if nom not in interdits:
        legitimes.append(nom)

print(legitimes)  # ["alice", "bob", "claire"]

```

Supprimer les doublons

Cas courant en analyse de journaux : une même adresse IP apparaît des dizaines de fois et l'on veut la liste des adresses distinctes.

```

releve = ["10.0.0.5", "10.0.0.7", "10.0.0.5", "10.0.0.9", "10.0.0.7"]
uniques = []

for ip in releve:
    if ip not in uniques:
        uniques.append(ip)

print(uniques)  # ["10.0.0.5", "10.0.0.7", "10.0.0.9"]

```

Application : reprendre une liste d'accès

Mise en situation — vous reprenez l'administration d'un serveur. La liste des comptes autorisés contient des comptes de test laissés par un prestataire, et des doublons.

```
acces = ["alice", "test", "bob", "alice", "guest", "claire", "bob"]
a_retirer = ["test", "guest"]

propre = []
for nom in acces:
    if nom not in a_retirer and nom not in propre:
        propre.append(nom)

propre.append("service_sauvegarde")
print("Comptes conservés :", len(propre))
print(propre)
```

Erreurs fréquentes

- Accéder à **liste[len(liste)]** : le dernier indice valide est **len - 1**.
- Supprimer des éléments pendant le parcours : des éléments sont sautés silencieusement.
- Croire que **b = a** crée une copie. Il faut **a.copy()** ou **list(a)**.
- Confondre **append**, qui ajoute un élément, et **extend**, qui ajoute tous les éléments d'une autre liste.
- Utiliser **remove** sur une valeur absente : cela déclenche une erreur.

Exercices

- À partir d'un relevé d'IP, produire la liste des adresses uniques et compter combien ont été retirées.
- Filtrer une liste de mots de passe pour ne garder que ceux d'au moins douze caractères.
- Écrire une fonction qui fusionne deux listes d'utilisateurs sans créer de doublon.
- Expliquer par écrit pourquoi le filtrage par construction est plus sûr que la suppression en cours de parcours.

Fiche 03 • Programmation • Python

Les dictionnaires : associer et compter

Une liste répond à « quels éléments ». Un dictionnaire répond à « combien de fois » et « lequel correspond à quoi ». C'est l'outil de l'analyse de journaux.

Niveau	Intermédiaire — NSI terminale, BTS SIO
Prérequis	Listes, boucles, conditions
Durée	1 h 30 avec exercices

Objectifs d'apprentissage

- Créer un dictionnaire et accéder à une valeur par sa clé
- Ajouter, modifier et retirer une association
- Parcourir clés, valeurs et paires
- Compter des occurrences — le motif le plus utile du cours
- Éviter l'erreur de clé absente avec **get**

Le problème que résout un dictionnaire

Vous disposez d'un relevé de tentatives de connexion et vous voulez savoir quelle adresse revient le plus souvent. Avec une liste seule, il faut la reparcourir entièrement pour chaque adresse. Avec un dictionnaire, un seul passage suffit.

Un dictionnaire associe une **clé** à une **valeur**. Ici : une adresse IP à un nombre de tentatives.

```
tentatives = {"10.0.0.5": 3, "10.0.0.7": 41, "10.0.0.9": 1}

print(tentatives["10.0.0.7"])    # 41
print(len(tentatives))           # 3
```

Créer, modifier, retirer

```
roles = {}                                # dictionnaire vide

roles["alice"] = "admin"                  # ajout
roles["bob"] = "lecteur"
roles["bob"] = "editeur"                  # modification : la clé existe déjà

del roles["alice"]                        # suppression
print("bob" in roles)                     # True
print(roles)                             # {"bob": "editeur"}
```

Une clé est unique. Réaffecter une clé existante remplace la valeur, cela n'ajoute pas une seconde entrée.

Accéder sans risque avec get

Accéder à une clé absente provoque une erreur qui interrompt le programme. **get** renvoie une valeur par défaut à la place.

```
roles = {"bob": "editeur"}

# print(roles["carole"])          # KeyError : le programme s'arrête
print(roles.get("carole"))        # None
print(roles.get("carole", "aucun rôle")) # "aucun rôle"
```

Parcourir un dictionnaire

```
tentatives = {"10.0.0.5": 3, "10.0.0.7": 41, "10.0.0.9": 1}

for ip in tentatives:                # les clés
    print(ip)

for nombre in tentatives.values():    # les valeurs
    print(nombre)

for ip, nombre in tentatives.items(): # les deux
    print(ip, "->", nombre, "tentatives")
```

Le motif à retenir : compter des occurrences

C'est le schéma le plus réutilisable de toute la fiche. Il se retrouve dans l'analyse de journaux, le comptage de mots, la détection de valeurs anormales.

```
connexions = ["10.0.0.5", "10.0.0.7", "10.0.0.7",
              "10.0.0.5", "10.0.0.7", "10.0.0.9"]

compteur = {}
for ip in connexions:
    compteur[ip] = compteur.get(ip, 0) + 1

print(compteur)
# {"10.0.0.5": 2, "10.0.0.7": 3, "10.0.0.9": 1}
```

La ligne centrale se lit ainsi : *prends le compte actuel de cette adresse, ou zéro si elle est inconnue, ajoute un, et range le résultat*. Elle traite d'un seul coup le cas de la première apparition et celui des suivantes.

Application : repérer une attaque par force brute

Mise en situation — vous êtes analyste. On vous transmet les adresses ayant échoué à s'authentifier cette nuit. Une adresse qui dépasse largement les autres n'est pas un utilisateur distrait.

```
echecs = ["10.0.0.5", "203.0.113.9", "203.0.113.9", "10.0.0.7",
          "203.0.113.9", "203.0.113.9", "10.0.0.5"]
SEUIL = 3

compteur = {}
for ip in echecs:
    compteur[ip] = compteur.get(ip, 0) + 1

for ip, n in compteur.items():
    if n >= SEUIL:
        print("ALERTE", ip, ":", n, "échecs")
    else:
        print("normal ", ip, ":", n)
```

Question à poser aux élèves : que se passe-t-il si l'attaquant répartit ses tentatives sur cinquante adresses différentes ? C'est la limite d'un seuil par adresse, et le point de départ d'une discussion sur la détection.

Erreurs fréquentes

- Accéder à une clé absente avec les crochets au lieu de **get** : le programme s'arrête sur une erreur.
- Utiliser une liste comme clé : seules les valeurs non modifiables peuvent être des clés.
- Croire qu'un dictionnaire peut contenir deux fois la même clé.
- Oublier **.items()** et tenter de débiller directement une clé en deux variables.
- Modifier le dictionnaire pendant qu'on le parcourt.

Exercices

- Compter les occurrences de chaque code de réponse dans une liste de statuts HTTP.
- Construire un annuaire associant un nom d'utilisateur à son rôle, puis afficher tous les administrateurs.
- À partir d'un compteur, trouver la clé ayant la plus grande valeur sans utiliser de fonction toute faite.
- Expliquer par écrit pourquoi le comptage par dictionnaire est plus efficace qu'un double parcours de liste.

Fiche 04 · Programmation · Python appliqué

Lire un fichier de journal

Dernière étape du parcours : les boucles, les listes et les dictionnaires réunis sur une donnée réelle. Un fichier de journal d'authentification, et trois questions à lui poser.

Niveau	Intermédiaire — NSI terminale, BTS SIO
Prérequis	Boucles, listes, dictionnaires
Durée	2 h avec exercices

Objectifs d'apprentissage

- Ouvrir et fermer un fichier proprement avec **with**
- Parcourir un fichier ligne par ligne sans tout charger en mémoire
- Découper une ligne en champs avec **split**
- Nettoyer une chaîne avec **strip**
- Combiner les trois notions précédentes pour produire une synthèse

Ouvrir un fichier correctement

Un fichier ouvert doit être refermé. Le mot-clé **with** s'en charge automatiquement, y compris si une erreur survient au milieu du traitement.

```
with open("auth.log", "r", encoding="utf-8") as f:
    for ligne in f:
        print(ligne.strip())
```

- **"r"** ouvre en lecture seule. C'est le mode à utiliser tant qu'on ne veut rien modifier.
- **encoding="utf-8"** évite les caractères accentués illisibles. À indiquer systématiquement.
- **for ligne in f** lit le fichier ligne par ligne : un fichier d'un gigaoctet ne sature pas la mémoire.
- **strip()** retire le retour à la ligne final et les espaces en trop.

Piège classique : sans strip(), chaque ligne se termine par un caractère de saut de ligne invisible. Les comparaisons échouent alors sans raison apparente.

Découper une ligne en champs

Un journal est structuré. Chaque ligne suit le même format, et **split** permet d'en extraire les parties.

```
ligne = "2026-07-12 03:14:22 ECHEC alice 203.0.113.9"

champs = ligne.split(" ")
print(champs)
# ["2026-07-12", "03:14:22", "ECHEC", "alice", "203.0.113.9"]

date, heure, resultat, compte, ip = ligne.split(" ")
print(resultat, compte, ip)
```

Le déballage direct en cinq variables suppose exactement cinq champs. Sur une ligne malformée, il échoue. En production on vérifie d'abord la longueur.

Tout réunir : analyser le journal

Mise en situation — on vous transmet le journal d'authentification de la nuit. Trois questions : combien d'échecs, quelles adresses en sont responsables, laquelle dépasse le seuil d'alerte.

```
SEUIL = 5
total_echecs = 0
par_ip = {}

with open("auth.log", "r", encoding="utf-8") as f:
    for ligne in f:
        champs = ligne.strip().split(" ")
        if len(champs) != 5:
            continue # ligne malformée, on passe
        resultat = champs[2]
        ip = champs[4]
        if resultat == "ECHEC":
            total_echecs = total_echecs + 1
            par_ip[ip] = par_ip.get(ip, 0) + 1

print("Échecs au total :", total_echecs)
print("Adresses distinctes :", len(par_ip))

for ip, n in par_ip.items():
    if n >= SEUIL:
        print("ALERTE", ip, "->", n, "échecs")
```

Ce programme fait tenir en vingt lignes ce que les trois fiches précédentes ont préparé : une boucle pour parcourir, une condition pour filtrer, un dictionnaire pour compter. C'est le point d'arrivée du parcours.

Aller plus loin

- Extraire l'heure et regarder si les échecs se concentrent sur une plage horaire.
- Comparer le nombre d'échecs et de succès d'une même adresse : un succès après quarante échecs n'a pas le même sens qu'un succès isolé.
- Écrire le résultat dans un fichier de synthèse plutôt que de l'afficher.
- Discuter des limites : que voit-on, que ne voit-on pas dans un journal ?

Erreurs fréquentes

- Oublier **strip()** : le saut de ligne fausse toutes les comparaisons.
- Utiliser **f.read()** sur un gros fichier : tout est chargé en mémoire d'un seul coup.
- Omettre **encoding** : les accents deviennent illisibles selon la machine.
- Déballer un nombre fixe de champs sans vérifier la longueur de la ligne.
- Ouvrir en mode "**w**" par inadvertance : le fichier est écrasé immédiatement.

Exercices

- Compter les connexions réussies par utilisateur.
- Trouver l'heure à laquelle les échecs sont les plus nombreux.
- Repérer les adresses ayant échoué puis réussi dans la même nuit.
- Écrire le résultat de l'analyse dans un fichier de synthèse.
- Rédiger en trois phrases ce que ce journal ne permet pas de savoir.